

Dưới đây là phiên bản dịch tiếng Việt của văn bản Markdown:

Nội dung: Được cấp phép bởi Google, nếu có sự công nhận phù hợp, việc sao chép các bảng và hình vẽ trong bài viết này chỉ dành riêng cho mục đích sử dụng trong các tác phẩm báo chí hoặc học thuật.

Chú ý là tất cả những gì bạn cần

Ashish Vaswani* **Noam Shazeer*** **Niki Parmar*** **Jakob Uszkoreit*** Google Brain Google Brain Google Research Google Research avaswani@google.com noam@google.com nikip@google.com usz@google.com

Llion Jones* **Aidan N. Gomez*†** **Lukasz Kaiser*** Google Research Đại học Toronto Google Brain llion@google.com aidan@cs.toronto.edu lukaszkaiser@google.com

Illia Polosukhin*‡ illia.polosukhin@gmail.com

Tóm tắt

Các mô hình chuyển đổi chuỗi hàng đầu dựa trên các mạng nơ-ron tái phát hoặc tích chập phức tạp bao gồm cả mã hóa và giải mã. Các mô hình hoạt động tốt nhất cũng kết nối mã hóa và giải mã thông qua một cơ chế chú ý. Chúng tôi đề xuất một kiến trúc mạng mới đơn giản, gọi là Biến đổi, dựa hoàn toàn vào các cơ chế chú ý, bỏ qua tái phát và tích chập hoàn toàn. Các thí nghiệm trên hai nhiệm vụ dịch máy cho thấy các mô hình này vượt trội về chất lượng trong khi dễ dàng song song hóa hơn và đòi hỏi ít thời gian huấn luyện hơn đáng kể. Mô hình của chúng tôi đạt được 28.4 điểm BLEU trên nhiệm vụ dịch tiếng Anh sang tiếng Đức của WMT 2014, cải thiện hơn kết quả hiện tại tốt nhất, bao gồm cả các nhóm, hơn 2 điểm BLEU. Trên nhiệm vụ dịch tiếng Anh sang tiếng Pháp của WMT 2014, mô hình của chúng tôi thiết lập điểm BLEU tốt nhất hiện tại cho một mô hình đơn là 41.8 sau khi huấn luyện trong 3.5 ngày trên tám GPU, một phần nhỏ chi phí huấn luyện của các mô hình tốt nhất từ các nghiên cứu trước đây. Chúng tôi chứng minh rằng mô hình Biến đổi tổng quát hóa tốt với các nhiệm vụ khác bằng cách áp dụng nó thành công vào phân tích cú pháp cấu trúc tiếng Anh, cả với dữ liệu huấn luyện lớn và giới hạn.

* Tham gia bình đẳng. Thứ tự liệt kê ngẫu nhiên. Jakob đề xuất thay thế mạng nơ-ron tái phát bằng tự chú ý và bắt đầu nỗ lực đánh giá ý tưởng này. Ashish, cùng Illia, thiết kế và thực hiện các mô hình Biến đổi đầu tiên và đã đóng vai trò quan trọng trong mọi khía cạnh của công việc này.

Noam đề xuất chú ý tích vô hướng đã chuẩn hóa, chú ý đa đầu và biểu diễn vị trí không tham số và trở thành người thứ hai tham gia gần như mọi chi tiết. Niki thiết kế, thực hiện, điều chỉnh và đánh giá vô số biến thể mô hình trong cơ sở mã gốc của chúng tôi và tensor2tensor. Llion cũng thử nghiệm với các biến thể mô hình mới, chịu trách nhiệm cho cơ sở mã gốc ban đầu của chúng tôi, và suy luận hiệu quả và hình ảnh hóa. Łukasz và Aidan đã dành nhiều ngày dài thiết kế và thực hiện các phần khác nhau của tensor2tensor, thay thế cơ sở mã gốc trước đó của chúng tôi, cải thiện đáng kể kết quả và tăng tốc nghiên cứu của chúng tôi một cách mạnh mẽ.

† Công việc được thực hiện khi ở Google Brain.

‡ Công việc được thực hiện khi ở Google Research.

Hội nghị lần thứ 31 về Xử lý Thông tin Thần kinh (NIPS 2017), Long Beach, California, Hoa Kỳ.

1 Giới thiệu

Mạng nơ-ron tái phát, hồi quy ngắn hạn dài hạn [13] và đơn vị tái phát có van [7] đặc biệt, đã được xác lập vững chắc như các phương pháp hàng đầu trong mô hình hóa chuỗi và các vấn đề chuyển đổi như mô hình ngôn ngữ và dịch máy [35, 2, 5]. Nhiều nỗ lực tiếp tục từ đó đã cố gắng mở rộng ranh giới của các mô hình ngôn ngữ tái phát và kiến trúc mã hóa-giải mã [38, 24, 15].

Mô hình tái phát thường phân tách tính toán theo vị trí ký tự của chuỗi đầu vào và đầu ra. Đối sánh vị trí với bước trong thời gian tính toán, chúng tạo ra một chuỗi các trạng thái tiềm ẩn ht , như một hàm của trạng thái tiềm ẩn trước đó $ht-1$ và đầu vào cho vị trí t . Tính chất tự nhiên tuần tự này ngăn chặn song song hóa bên trong các ví dụ huấn luyện, điều trở nên quan trọng hơn ở độ dài chuỗi dài hơn, do các ràng buộc bộ nhớ hạn chế việc ghép các ví dụ. Các công trình gần đây đã đạt được cải tiến đáng kể về hiệu quả tính toán thông qua các thủ thuật phân tách [21] và tính toán điều kiện [32], đồng thời cũng cải thiện hiệu suất mô hình trong trường hợp sau. Tuy nhiên, ràng buộc cơ bản của tính toán tuần tự vẫn còn tồn tại.

Các cơ chế chú ý đã trở thành một phần không thể thiếu của các mô hình mô hình hóa chuỗi và chuyển đổi hấp dẫn trong nhiều nhiệm vụ, cho phép mô hình hóa các mối quan hệ phụ thuộc mà không cần quan tâm đến khoảng cách của chúng trong chuỗi đầu vào hoặc đầu ra [2, 19]. Tuy nhiên, trong hầu hết các trường hợp [27], các cơ chế chú ý này được sử dụng cùng với một mạng nơ-ron tái phát.

Trong công trình này, chúng tôi đề xuất mô hình Biến đổi, một kiến trúc mô hình từ bỏ

tính toán tuần tự và thay vào đó dựa hoàn toàn vào một cơ chế chú ý để tạo ra các mối quan hệ toàn cục giữa đầu vào và đầu ra. Mô hình Biến đổi cho phép song song hóa nhiều hơn đáng kể và có thể đạt được trạng thái hàng đầu mới trong chất lượng dịch sau khi huấn luyện trong ít nhất 12 giờ trên 8 GPU P100.

Mục tiêu giảm thiểu tính toán tuần tự cũng tạo nền tảng cho Extended Neural GPU [16], ByteNet [18] và ConvS2S [9], tất cả đều sử dụng mạng nơ-ron tích chập như khối xây dựng cơ bản, tính toán biểu diễn ẩn song song cho tất cả vị trí đầu vào và đầu ra. Trong các mô hình này, số lượng phép tính cần thiết để liên kết tín hiệu từ hai vị trí đầu vào hoặc đầu ra tùy ý tăng theo khoảng cách giữa các vị trí, tuyến tính đối với ConvS2S và logarit đối với ByteNet. Điều này khiến việc học phụ thuộc giữa các vị trí xa nhau trở nên khó khăn hơn [12]. Trong Transformer, điều này được giảm xuống còn một số lượng cố định các phép tính, mặc dù với chi phí giảm độ phân giải hiệu quả do trung bình hóa vị trí được cân bằng bởi trọng số chú ý, một hiệu ứng mà chúng tôi khắc phục bằng Cơ chế Chú ý Đa Đầu như được mô tả trong phần 3.2.

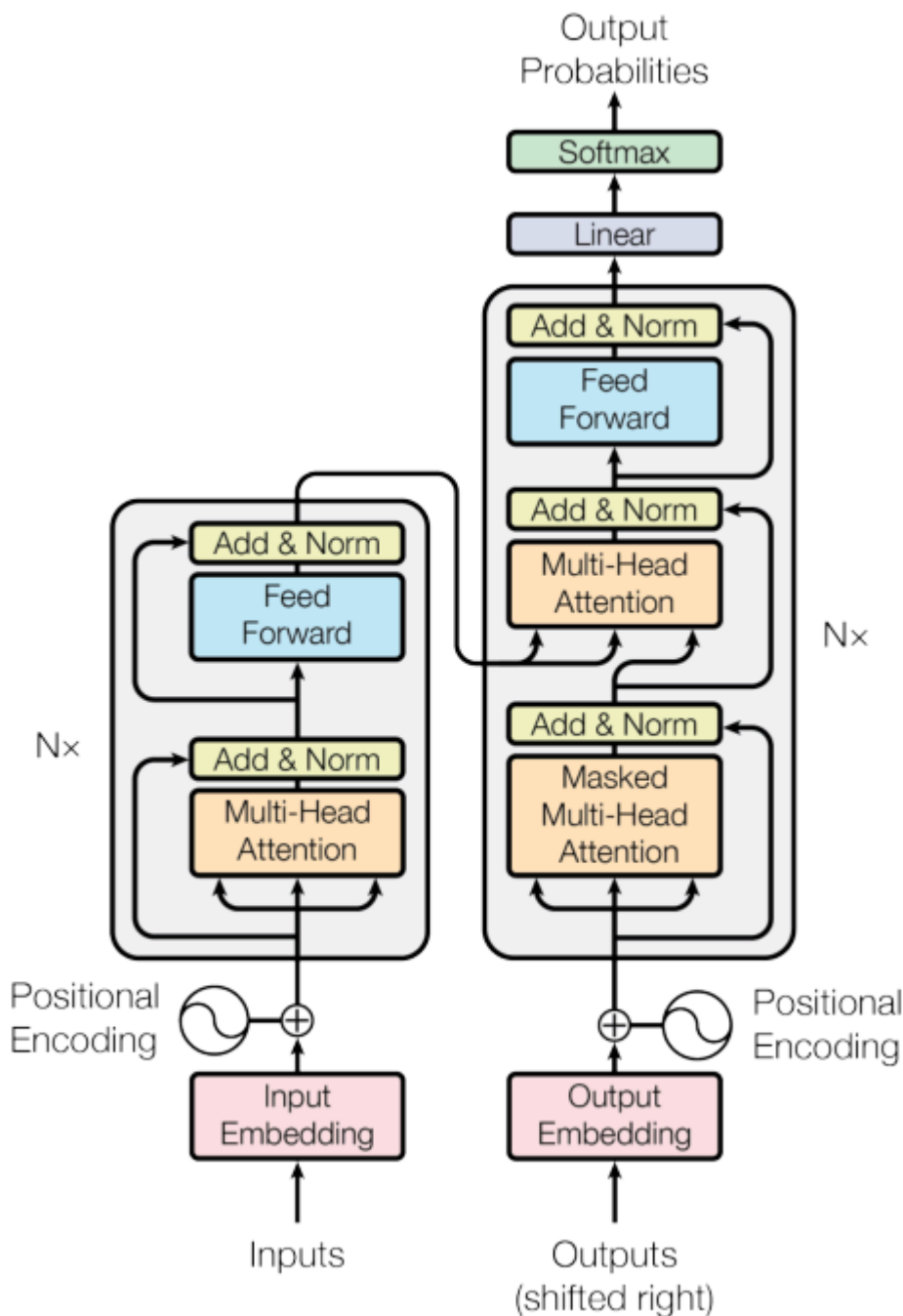
Chú ý nội tại, đôi khi gọi là chú ý nội bộ, là một cơ chế chú ý liên kết các vị trí khác nhau của một chuỗi đơn để tính toán biểu diễn của chuỗi đó. Chú ý nội tại đã được sử dụng thành công trong nhiều nhiệm vụ khác nhau, bao gồm hiểu nội dung văn bản, tóm tắt trừu tượng, sự ràng buộc văn bản và học biểu diễn câu độc lập nhiệm vụ [4, 27, 28, 22].

Mạng nhớ cuối cùng đến cuối cùng dựa trên một cơ chế chú ý tái phát thay vì tái phát theo trình tự và đã được chứng minh là hoạt động tốt trên các nhiệm vụ trả lời câu hỏi với ngôn ngữ đơn giản và mô hình ngôn ngữ [34].

Theo kiến thức của chúng tôi, tuy nhiên, Transformer là mô hình chuyển dịch đầu tiên hoàn toàn dựa vào chú ý nội tại để tính toán biểu diễn của đầu vào và đầu ra của nó mà không sử dụng mạng nơ-ron tái phát theo trình tự hoặc tích chập. Trong các phần tiếp theo, chúng tôi sẽ mô tả về Transformer, thuyết minh về chú ý nội tại và thảo luận về ưu điểm của nó so với các mô hình như [17, 18] và [9].

3 Kiến trúc Mô hình

Hầu hết các mô hình chuyển dịch chuỗi thần kinh cạnh tranh đều có cấu trúc mã hóa - giải mã [5, 2, 35]. Ở đây, mã hóa viên biến một chuỗi đầu vào của biểu diễn ký hiệu $(x_1, \dots, x_n)$ thành một chuỗi biểu diễn liên tục $\mathbf{z} = (z_1, \dots, z_n)$. Dựa trên $\mathbf{z}$, giải mã viên sau đó tạo ra một chuỗi đầu ra $(y_1, \dots, y_m)$ của ký hiệu một phần tử tại một thời điểm. Ở mỗi bước, mô hình là tự hồi quy [10], tiêu thụ các ký hiệu đã sinh ra trước đó như đầu vào bổ sung khi tạo ra phần tử tiếp theo.



Hình 1: Kiến trúc mô hình Transformer.

Transformer tuân theo cấu trúc tổng thể này sử dụng các lớp tự chú ý chồng lên và các lớp kết nối đầy đủ, điểm, cho cả mã hóa viên và giải mã viên, được hiển thị ở nửa bên trái và phải của Hình 1, tương ứng.

3.1 Các chồng mã hóa viên và giải mã viên

Mã hóa viên: Mã hóa viên được cấu thành từ một chồng $N = 6$ lớp giống nhau. Mỗi lớp có hai lớp con. Lớp con thứ nhất là cơ chế chú ý tự đa đầu, và lớp con thứ hai là mạng kết nối đầy đủ đơn giản, vị trí, điểm. Chúng tôi áp dụng một kết nối dư [11] xung quanh mỗi trong hai lớp con, sau đó là chuẩn hóa lớp [1]. Đó là, đầu ra của mỗi lớp con là $\text{LayerNorm}(x + \text{Sublayer}(x))$, nơi $\text{Sublayer}(x)$ là hàm được thực hiện bởi lớp con chính nó.

Để hỗ trợ các kết nối dư này, tất cả các lớp con trong mô hình, cũng như các lớp biểu diễn, sản xuất đầu ra của chiều $d_{model} = 512$.

Giải mã viên: Giải mã viên cũng được cấu thành từ một chồng $N = 6$ lớp giống nhau. Ngoài hai lớp con trong mỗi lớp mã hóa viên, giải mã viên chèn một lớp con thứ ba, thực hiện chú ý tự đa đầu trên đầu ra của chồng mã hóa viên. Tương tự như mã hóa viên, chúng tôi áp dụng các kết nối dư xung quanh mỗi lớp con, sau đó là chuẩn hóa lớp. Chúng tôi cũng sửa đổi lớp con chú ý tự trong chồng giải mã viên để ngăn chặn vị trí từ chú ý đến các vị trí sau. Sự che phủ này, kết hợp với thực tế rằng các biểu diễn đầu ra bị lệch một vị trí, đảm bảo rằng dự đoán cho vị trí i chỉ có thể phụ thuộc vào các đầu ra đã biết tại các vị trí nhỏ hơn i .

3.2 Chú ý

Một hàm chú ý có thể được mô tả như ánh xạ một truy vấn và một tập hợp các cặp khóa-giá trị đến một đầu ra, nơi truy vấn, khóa, giá trị và đầu ra đều là vector. Đầu ra được tính như một tổng trọng số của các giá trị, nơi trọng số được gán cho mỗi giá trị được tính bằng một hàm tương thích của truy vấn với khóa tương ứng.

3.2.1 Chú ý Tích Vô Hướng Đã Chuẩn Hóa

Chúng tôi gọi chú ý cụ thể của mình là "Chú ý Tích Vô Hướng Đã Chuẩn Hóa" (Hình 2). Đầu vào bao gồm các truy vấn và khóa của chiều d_k , và giá trị của chiều d_v . Chúng tôi tính toán tích vô hướng của truy vấn với tất cả các khóa, chia mỗi bằng $\sqrt{d_k}$, và áp dụng hàm softmax để thu được trọng số trên các giá trị.

Trong thực hành, chúng tôi tính toán hàm chú ý trên một tập hợp các truy vấn đồng thời, đóng gói lại thành ma trận Q . Khóa và giá trị cũng được đóng gói lại thành ma trận K và V . Chúng tôi tính toán ma trận đầu ra như:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (1)$$

Hai hàm chú ý được sử dụng phổ biến nhất là chú ý cộng thêm [2], và chú ý tích vô hướng (nhân). Chú ý tích vô hướng tương đương với thuật toán của chúng tôi, ngoại trừ hệ số chuẩn hóa $\sqrt{d_k}$. Chú ý cộng thêm tính toán hàm tương thích bằng mạng kết nối đầy đủ với một lớp ẩn duy nhất. Trong lý thuyết, hai hàm này tương tự về độ phức tạp, nhưng chú ý tích vô hướng nhanh hơn và tiết kiệm không gian hơn trong thực hành, vì nó có thể được thực hiện bằng mã nhân ma trận tối ưu hóa cao.

Dịch đoạn văn bản Markdown từ tiếng Anh sang tiếng Việt:

Text: Trong trường hợp giá trị nhỏ của dk , hai cơ chế hoạt động tương tự nhau, nhưng chú ý cộng thêm vượt trội hơn so với chú ý tích vô hướng mà không cần chuẩn hóa cho các giá trị lớn của dk [3]. Chúng tôi nghi ngờ rằng đối với các giá trị lớn của dk , các tích vô hướng tăng lên đáng kể về độ lớn, đẩy hàm softmax vào các vùng có độ dốc cực kỳ nhỏ⁴. Để khắc phục hiệu ứng này, chúng tôi chuẩn hóa các tích vô hướng bằng $\frac{1}{\sqrt{d_k}}$.

3.2.2 Chú ý đa đầu

Thay vì thực hiện một hàm chú ý duy nhất với d chiều khóa, giá trị và câu hỏi, chúng tôi tìm thấy lợi ích khi tuyến tính hóa các câu hỏi, khóa và giá trị h lần với các phép chiếu tuyến tính khác nhau, học được đến dk , dk và dv chiều tương ứng. Trên mỗi phiên bản đã chiếu của khóa, câu hỏi và giá trị, chúng tôi sau đó thực hiện hàm chú ý song song, tạo ra các giá trị dv -chiều.

⁴Để minh họa tại sao các tích vô hướng tăng lên, hãy giả định rằng các thành phần của q và k là các biến ngẫu nhiên độc lập với trung bình 0 và phương sai 1. Thì tích vô hướng của chúng, $q \cdot k = \sum_{i=1}^d q_i k_i$, có trung bình 0 và phương sai d .

4

giá trị đầu ra. Những giá trị này được nối tiếp và một lần nữa được chiếu, dẫn đến các giá trị cuối cùng, như thể hiện trong Hình 2.

Chú ý đa đầu cho phép mô hình đồng thời chú ý đến thông tin từ các không gian biểu diễn phụ khác nhau ở các vị trí khác nhau. Với một đầu chú ý duy nhất, trung bình hóa ngăn chặn điều này.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$\text{where head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Trong đó các phép chiếu là ma trận tham số $W_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}$, $W_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ và $W^O \in \mathbb{R}^{hd_v \times d_{\text{model}}}$.

Trong công trình này, chúng tôi sử dụng $h = 8$ lớp chú ý song song, hoặc đầu. Đối với mỗi lớp này, chúng tôi sử dụng $dk = dv = d_{\text{model}} / h = 64$. Do chiều kích giảm của mỗi đầu, tổng chi phí tính toán tương tự như của chú ý đơn đầu với chiều kích đầy đủ.

3.2.3 Ứng dụng của Chú ý trong Mô hình của chúng tôi

Transformer sử dụng chú ý đa đầu theo ba cách khác nhau:

- Trong các lớp "chú ý mã hóa-giải mã", các câu hỏi đến từ lớp giải mã trước đó, và các khóa nhớ và giá trị đến từ đầu ra của lớp mã hóa. Điều này cho phép mỗi vị trí trong giải mã chú ý đến tất cả các vị trí trong chuỗi đầu vào. Điều này mô phỏng các cơ chế chú ý mã hóa-giải mã điển hình trong các mô hình chuyển đổi chuỗi như [38, 2, 9].
- Mã hóa chứa các lớp chú ý tự. Trong một lớp chú ý tự, tất cả các khóa, giá trị và câu hỏi đều đến từ cùng một nơi, trong trường hợp này, đầu ra của lớp trước đó trong mã hóa. Mỗi vị trí trong mã hóa có thể chú ý đến tất cả các vị trí trong lớp trước đó của mã hóa.
- Tương tự, các lớp chú ý tự trong giải mã cho phép mỗi vị trí trong giải mã chú ý đến tất cả các vị trí trong giải mã lên đến và bao gồm vị trí đó. Chúng tôi cần ngăn chặn luồng thông tin trái chiều trong giải mã để bảo tồn tính chất tự hồi quy. Chúng tôi thực hiện điều này bên trong chú ý tích vô hướng đã chuẩn hóa bằng cách che phủ (đặt thành $-\infty$) tất cả các giá trị trong đầu vào của hàm softmax tương ứng với các kết nối bất hợp pháp. Xem Hình 2.

3.3 Mạng lưới Feed-Forward theo Vị Trí

Ngoài các lớp chú ý, mỗi lớp trong mã hóa và giải mã của chúng tôi chứa một mạng lưới feed-forward hoàn toàn kết nối, được áp dụng riêng biệt và đồng nhất cho mỗi vị trí. Điều này bao gồm hai phép biến đổi tuyến tính với một hàm kích hoạt ReLU giữa.

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad (2)$$

Trong khi các phép biến đổi tuyến tính giống nhau qua các vị trí khác nhau, chúng sử dụng các tham số khác nhau từ lớp này sang lớp khác. Một cách khác để mô tả điều này là hai phép chiếu với kích thước hạt 1. Chiều kích của đầu vào và đầu ra là $d_{\text{model}} = 512$, và lớp giữa có chiều kích $d_{\text{ff}} = 2048$.

3.4 Embeddings và Softmax

Tương tự như các mô hình chuyển đổi chuỗi khác, chúng tôi sử dụng các embeddings học được để chuyển đổi các token đầu vào và đầu ra thành các vector chiều d_{model} . Chúng tôi cũng sử dụng phép biến đổi tuyến tính học được thông thường và hàm softmax để chuyển đổi đầu ra của giải mã thành xác suất token tiếp theo dự đoán. Trong mô hình của chúng tôi, chúng tôi chia sẻ cùng một ma trận trọng số giữa hai lớp embeddings và phép biến đổi tuyến tính trước softmax, tương tự như [30]. Tr

Chúng tôi cũng đã thử nghiệm việc sử dụng vị trí nhúng học được [9] thay vì phương pháp truyền thống, và phát hiện ra rằng hai phiên bản này tạo ra kết quả gần như tương

đường (xem hàng (E) trong Bảng 3). Chúng tôi chọn phiên bản sin vì nó có thể cho phép mô hình dự đoán chuỗi dài hơn những chuỗi gặp phải trong quá trình huấn luyện.

4 Tại sao nên sử dụng Self-Attention

Trong phần này, chúng tôi so sánh các khía cạnh khác nhau của lớp self-attention với các lớp tái phát và lớp tích chập thường được sử dụng để ánh xạ một chuỗi biểu diễn ký hiệu có độ dài biến đổi ($x_1, \dots, x_n$) sang một chuỗi biểu diễn có độ dài tương đương ($z_1, \dots, z_n$), với $x_i, z_i \in \mathbb{R}^d$, ví dụ như một lớp ẩn trong một bộ mã hóa hoặc bộ giải mã truyền thống của quá trình chuyển đổi chuỗi. Để thúc đẩy việc sử dụng self-attention, chúng tôi cân nhắc ba yếu tố mong muốn.

Một là tổng độ phức tạp tính toán mỗi lớp. Hai là lượng tính toán có thể song song hóa, được đo bằng số lượng tối thiểu các phép tính tuần tự cần thiết.

Thứ ba là chiều dài đường giữa các phụ thuộc xa trong mạng. Học các phụ thuộc xa là thách thức chính trong nhiều nhiệm vụ chuyển đổi chuỗi. Một yếu tố chính ảnh hưởng đến khả năng học các phụ thuộc này là chiều dài của các đường đi trước và sau mà tín hiệu phải đi qua trong mạng. Các đường đi ngắn hơn giữa bất kỳ sự kết hợp nào của vị trí đầu vào và đầu ra sẽ giúp dễ dàng học các phụ thuộc xa hơn [12]. Do đó, chúng tôi cũng so sánh chiều dài đường lớn nhất giữa bất kỳ hai vị trí đầu vào và đầu ra nào trong các mạng được tạo thành từ các loại lớp khác nhau.

Như được ghi chú trong Bảng 1, một lớp self-attention kết nối tất cả các vị trí với một số lượng cố định các phép tính tuần tự, trong khi một lớp tái phát yêu cầu $O(n)$ phép tính tuần tự. Về độ phức tạp tính toán, các lớp self-attention nhanh hơn các lớp tái phát khi độ dài chuỗi n nhỏ hơn chiều dài biểu diễn d , điều này thường xảy ra với các biểu diễn câu được sử dụng bởi các mô hình tiên tiến trong dịch máy, như biểu diễn word-piece [38] và byte-pair [31].

Để cải thiện hiệu suất tính toán cho các nhiệm vụ liên quan đến chuỗi rất dài, self-attention có thể bị giới hạn trong việc chỉ xét một khu vực lân cận có kích thước r trong chuỗi đầu vào tập trung xung quanh vị trí đầu ra tương ứng. Điều này sẽ tăng chiều dài đường lớn nhất lên $O(n/r)$. Chúng tôi dự định tiếp tục nghiên cứu phương pháp này trong công việc tương lai.

Một lớp tích chập đơn với chiều rộng nhân $k < n$ không kết nối tất cả các cặp vị trí đầu vào và đầu ra. Việc làm điều này đòi hỏi một chồng $O(n/k)$ lớp tích chập trong trường hợp nhân liên tục, hoặc $O(\log_k(n))$ trong trường hợp tích chập giãn nở [18], tăng chiều dài của đường dài nhất giữa bất kỳ hai vị trí nào trong mạng. Các lớp tích chập thường đắt hơn các lớp tái phát, với tỷ lệ k . Tuy nhiên, các lớp tích chập tách biệt [6] giảm đáng kể độ

phức tạp xuống $O(k \cdot n \cdot d + n \cdot d^2)$. Ngay cả khi $k = n$, độ phức tạp của một lớp tích chập tách biệt vẫn bằng sự kết hợp của một lớp self-attention và một lớp feed-forward điểm, cách tiếp cận mà chúng tôi sử dụng trong mô hình của mình.

Như lợi ích phụ, self-attention có thể tạo ra các mô hình dễ hiểu hơn. Chúng tôi kiểm tra phân phối chú ý từ các mô hình của mình và trình bày và thảo luận các ví dụ trong phụ lục. Không chỉ các đầu chú ý riêng lẻ rõ ràng học để thực hiện các công việc khác nhau, mà còn nhiều trong số chúng xuất hiện có hành vi liên quan đến cấu trúc cú pháp và ngữ nghĩa của các câu.

5 Huấn luyện

Khuôn khổ này mô tả quy trình huấn luyện cho các mô hình của chúng tôi.

5.1 Dữ liệu huấn luyện và Batch

Chúng tôi huấn luyện trên tập dữ liệu chuẩn WMT 2014 tiếng Anh - Đức, bao gồm khoảng 4,5 triệu cặp câu. Các câu được mã hóa bằng phương pháp byte-pair encoding [3], với từ điển chung giữa nguồn và mục tiêu chứa khoảng 37.000 token. Đối với tiếng Anh - Pháp, chúng tôi sử dụng tập dữ liệu WMT 2014 lớn hơn nhiều, bao gồm 36 triệu câu và chia token thành một từ điển 32.000 token [38]. Các cặp câu được nhóm lại dựa trên độ dài gần đúng của chuỗi. Mỗi batch huấn luyện chứa một tập hợp các cặp câu với khoảng 25.000 token nguồn và 25.000 token mục tiêu.

5.2 Thiết bị và Lịch trình

Chúng tôi huấn luyện các mô hình trên một máy tính với 8 GPU NVIDIA P100. Với các siêu tham số được mô tả trong toàn bộ bài viết cho các mô hình cơ bản, mỗi bước huấn luyện mất khoảng 0,4 giây. Chúng tôi huấn luyện các mô hình cơ bản tổng cộng 100.000 bước hoặc 12 giờ. Đối với các mô hình lớn (mô tả ở hàng cuối cùng của bảng 3), thời gian mỗi bước là 1,0 giây. Các mô hình lớn được huấn luyện 300.000 bước (3,5 ngày).

5.3 Tối ưu hóa

Chúng tôi sử dụng tối ưu hóa Adam [20] với $\beta_1 = 0.9$, $\beta_2 = 0.98$ và $\epsilon = 10^{-9}$. Chúng tôi điều chỉnh tốc độ học trong quá trình huấn luyện theo công thức:

$$lrate = d_{\text{model}}^{-0.5} \cdot \min(step_num^{-0.5}, step_num \cdot warmup_steps^{-1.5}) \quad (3)$$

Điều này tương ứng với việc tăng tốc độ học tuyến tính trong *warmup* bước đầu tiên và giảm nó theo tỷ lệ nghịch bình phương của số bước sau đó. Chúng tôi sử dụng *warmup* = 4000.

5.4 Quy tắc thường xuyên

Chúng tôi áp dụng ba loại quy tắc thường xuyên trong quá trình huấn luyện:

7

Bảng 2: Biến đổi đạt được điểm BLEU cao hơn so với các mô hình hiện đại nhất trước đây trên các bài kiểm tra mớistest2014 tiếng Anh - Đức và tiếng Anh - Pháp với chi phí huấn luyện thấp hơn đáng kể.

BLEU Chi phí huấn luyện (FLOPs)	--- --- --- ---	Mô hình	EN-DE EN-FR EN-DE EN-FR			
ByteNet [18]	23,75		Deep-Att + PosUnk [39]	39,2	1_0 · 10 ²⁰	
GNMT + RL [38]	24,6 39,92	2.3 · 10 ¹⁹ 1.4 · 10 ²⁰	ConvS2S [9]	25,16 40,46	9.6 · 10 ¹⁸	
1.5 · 10 ²⁰	MoE[32]	26,03 40,56	2.0 · 10 ¹⁹ 1.2 · 10 ²⁰	Deep-Att + PosUnk Ensemble [39]	40,4	8.0 · 10 ²⁰
GNMT + RL Ensemble [38]	26,30 41,16	1.8 · 10 ²⁰ 1.1 · 10 ²¹	ConvS2S Ensemble[9]	26,36	41,29	7.7 · 10 ¹⁹ 1.2 · 10 ²¹
Biến đổi (mô hình cơ bản)	27,3 38,1	3** · 3**** · **	10 ¹⁸	Biến đổi (lớn)	28,4 41,8	2.3 · 10 ¹⁹

Quy tắc thường xuyên dư lượng Chúng tôi áp dụng quy tắc thường xuyên [33] vào kết quả của mỗi lớp phụ trước khi cộng vào đầu vào của lớp phụ và chuẩn hóa. Ngoài ra, chúng tôi áp dụng quy tắc thường xuyên vào tổng của các biểu đồ và mã hóa vị trí trong cả hai chồng encoder và decoder. Đối với mô hình cơ bản, chúng tôi sử dụng tỷ lệ $P_{drop} = 0.1$.

Trừu tượng hóa nhãn Trong quá trình huấn luyện, chúng tôi áp dụng trừu tượng hóa nhãn có giá trị $\epsilon/s = 0.1$ [36]. Điều này làm giảm độ phức tạp, vì mô hình học cách trở nên chắc chắn hơn, nhưng cải thiện độ chính xác và điểm BLEU.

6 Kết quả

6.1 Dịch máy

Trên nhiệm vụ dịch từ tiếng Anh sang tiếng Đức của WMT 2014, mô hình biến đổi lớn (Transformer (big) trong bảng 2) vượt trội hơn so với các mô hình tốt nhất trước đây (bao gồm cả tập hợp mô hình) hơn 2.0 điểm BLEU, thiết lập một điểm chuẩn mới cho điểm số BLEU là 28.4. Cấu hình của mô hình này được liệt kê ở hàng cuối cùng của bảng 3. Thời gian huấn luyện mất 3.5 ngày trên 8 GPU P100. Ngay cả mô hình cơ sở của chúng ta cũng vượt trội hơn tất cả các mô hình và tập hợp mô hình trước đây, với chi phí huấn luyện ít hơn đáng kể so với bất kỳ mô hình cạnh tranh nào.

Trên nhiệm vụ dịch từ tiếng Anh sang tiếng Pháp của WMT 2014, mô hình lớn của chúng tôi đạt được điểm số BLEU là 41.0, vượt trội hơn tất cả các mô hình đơn lẻ đã công bố

trước đây, với chi phí huấn luyện ít hơn 1/4 so với mô hình điểm chuẩn hiện tại. Mô hình Transformer (big) được huấn luyện cho việc dịch từ tiếng Anh sang tiếng Pháp sử dụng tỷ lệ dropout $P_{drop} = 0.1$ thay vì 0.3.

Đối với các mô hình cơ sở, chúng tôi sử dụng một mô hình duy nhất được thu được bằng cách lấy trung bình 5 điểm kiểm tra cuối cùng, được viết vào mỗi 10 phút. Đối với các mô hình lớn, chúng tôi lấy trung bình 20 điểm kiểm tra cuối cùng. Chúng tôi sử dụng thuật toán tìm kiếm theo chiều rộng với kích thước chiều rộng là 4 và hệ số phạt độ dài $\alpha = 0.6$ [38]. Các siêu tham số này được chọn sau khi thử nghiệm trên tập phát triển. Chúng tôi đặt độ dài tối đa của đầu ra trong quá trình suy luận là độ dài đầu vào + 50, nhưng dừng sớm nếu có thể [38].

Bảng 2 tóm tắt kết quả của chúng tôi và so sánh chất lượng dịch và chi phí huấn luyện của chúng tôi với các kiến trúc mô hình khác từ các tài liệu. Chúng tôi ước tính số lượng phép tính dấu phẩy động được sử dụng để huấn luyện một mô hình bằng cách nhân thời gian huấn luyện, số lượng GPU được sử dụng và ước tính sức mạnh dấu phẩy động đơn chính xác duy trì của mỗi GPU⁵.

6.2 Các biến thể của mô hình

Để đánh giá tầm quan trọng của các thành phần khác nhau của mô hình biến đổi, chúng tôi đã thay đổi mô hình cơ sở của mình theo nhiều cách khác nhau, đo lường sự thay đổi về hiệu suất trên tập phát triển mới *test2013* của nhiệm vụ dịch từ tiếng Anh sang tiếng Đức.

⁵Chúng tôi sử dụng giá trị 2.8, 3.7, 6.0 và 9.5 TFLOPS cho K80, K40, M40 và P100 tương ứng.

8

Bảng 3: Các biến thể của kiến trúc mô hình biến đổi. Các giá trị chưa được liệt kê giống hệt với những giá trị của mô hình cơ sở. Tất cả các chỉ số đều dựa trên tập phát triển mới *test2013* của nhiệm vụ dịch từ tiếng Anh sang tiếng Đức. Các độ phức tạp được liệt kê theo mỗi token, theo mã hóa byte-pair của chúng tôi, và không nên so sánh với độ phức tạp theo mỗi từ.

	N	d_{model}	d_{ff}	h	dk	dv	P_{drop}	ϵ_{ls}	train steps	F (
base	6	512	2048	8	64	64	0.1	0.1	100K	4

				1	512	512				5
A				4	128	128				5
()				16	32	32				4
				32	16	16				5
					16					5
(B)					32					5
	2									6
	4									5
	8									4

Trong công trình này, chúng tôi giới thiệu Biến đổi (Transformer), mô hình đầu tiên dựa hoàn toàn trên Cơ chế chú ý (Attention Mechanism) cho phép chuyển đổi chuỗi, thay thế các lớp tái phát thường được sử dụng trong kiến trúc Mã hóa - Giải mã (Encoder-Decoder) bằng chú ý đa đầu (Multi-Head Attention).

Đối với các nhiệm vụ dịch máy, Biến đổi có thể được huấn luyện nhanh hơn đáng kể so với các kiến trúc dựa trên lớp tái phát hoặc lớp tích chập. Trên cả hai nhiệm vụ dịch WMT 2014 Tiếng Anh sang Tiếng Đức và WMT 2014 Tiếng Anh sang Tiếng Pháp, chúng tôi đạt được một kỷ lục mới về hiệu suất. Trong nhiệm vụ trước, mô hình tốt nhất của chúng tôi vượt qua cả các tập hợp mô hình đã báo cáo trước đây.

Chúng tôi rất hào hứng về tương lai của các mô hình dựa trên chú ý và kế hoạch áp dụng chúng vào các nhiệm vụ khác. Chúng tôi dự định mở rộng Biến đổi cho các vấn đề liên quan đến các mô hình đầu vào và đầu ra khác ngoài văn bản và điều tra các cơ chế chú ý cục bộ, bị hạn chế để xử lý hiệu quả các đầu vào và đầu ra lớn như hình ảnh, âm thanh và video. Làm cho quá trình sinh ra ít tuần tự hơn cũng là một mục tiêu nghiên cứu của chúng tôi.

Mã nguồn chúng tôi sử dụng để huấn luyện và đánh giá các mô hình của mình có sẵn tại <https://github.com/tensorflow/tensor2tensor>.

Tạ ơn: Chúng tôi cảm ơn Nal Kalchbrenner và Stephan Gouws vì những nhận xét, sửa

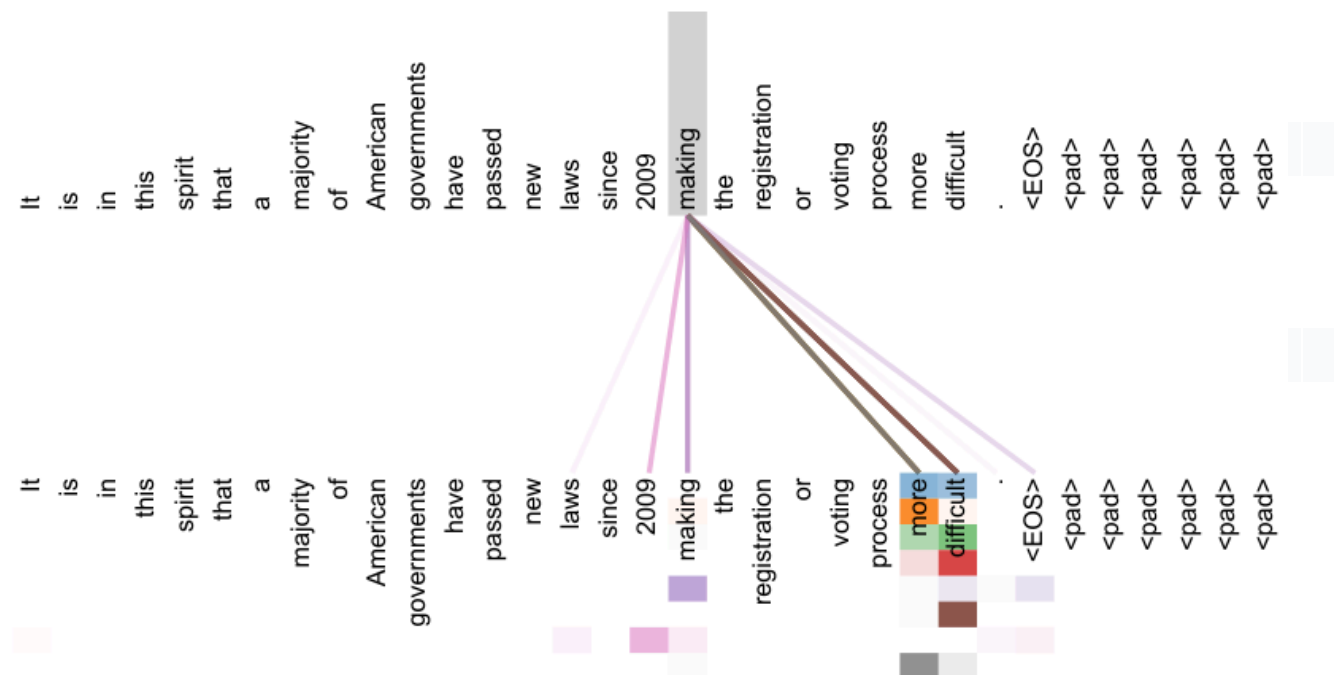
đổi và sự truyền cảm hứng hữu ích của họ.

Tham khảo:

[1] Jimmy Lei Ba, Jamie Ryan Kiros, và Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

- [2] Dzmitry Bahdanau, Kyunghyun Cho, và Yosh

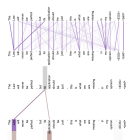
Hình thị cơ chế chú ý



Trong tinh thần đó, phần lớn chính phủ Hoa Kỳ đã ban hành các luật mới kể từ năm 2009 nhằm làm cho quá trình đăng ký hoặc bỏ phiếu trở nên khó khăn hơn.

Trong tinh thần đó, phần lớn chính phủ Hoa Kỳ đã ban hành các luật mới kể từ năm 2009 nhằm làm cho quá trình đăng ký hoặc bỏ phiếu trở nên khó khăn hơn.

Hình 3: Một ví dụ về cơ chế chú ý theo dõi sự phụ thuộc xa trong tự chú ý của mã hóa ở tầng thứ 5 trong tổng cộng 6 tầng. Nhiều đầu chú ý tập trung vào sự phụ thuộc xa của động từ 'làm', hoàn thiện cụm từ 'làm...nhiều hơn khó khăn'. Hình thị ở đây chỉ được hiển thị cho từ 'làm'. Màu sắc khác nhau đại diện cho các đầu chú ý khác nhau. Hình ảnh tốt nhất khi xem màu.



Luật pháp sẽ không bao giờ hoàn hảo, nhưng việc áp dụng nó phải công bằng - đó là

điều mà chúng ta đang thiếu, theo ý kiến cá nhân của tôi.

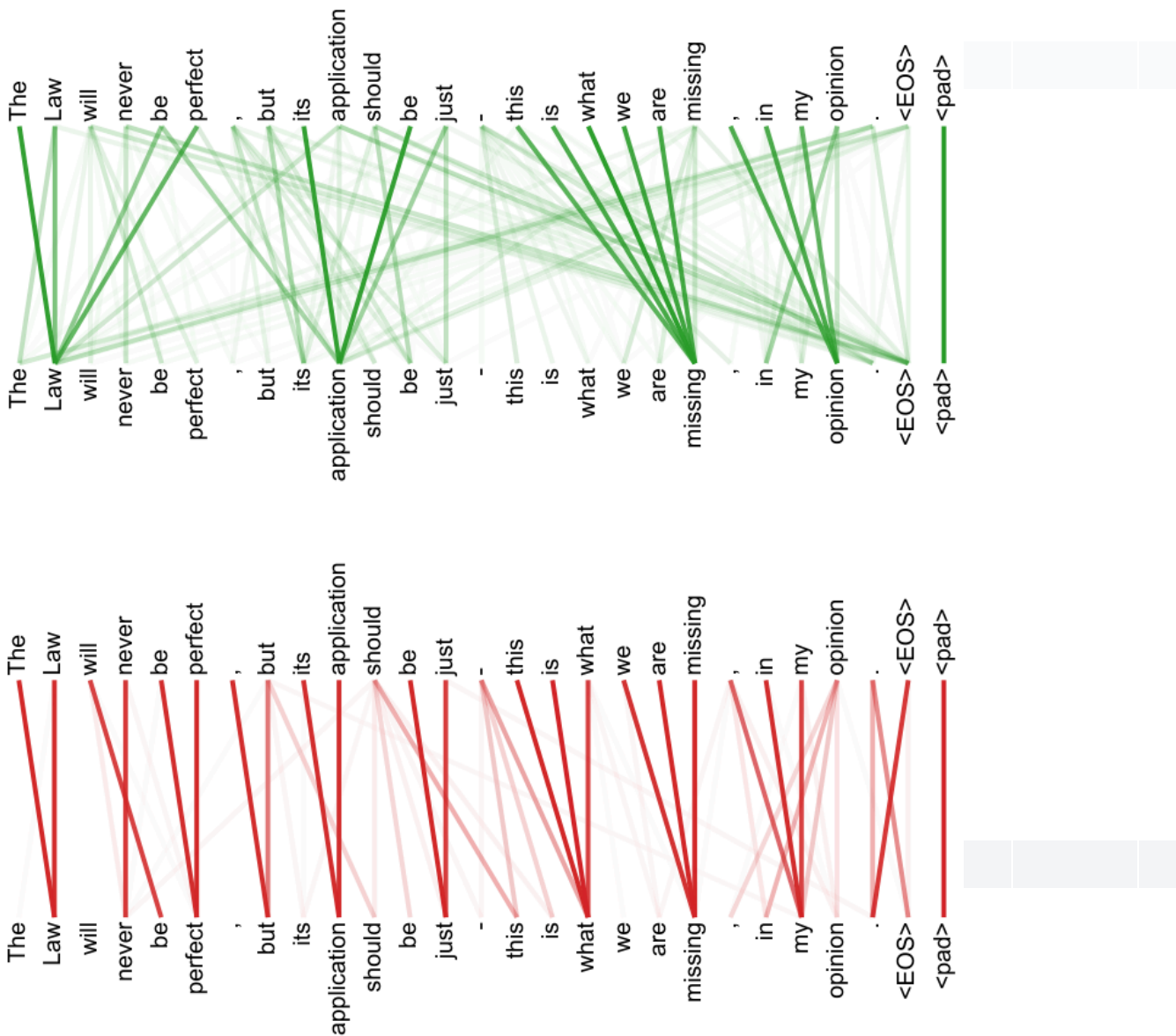
Luật pháp sẽ không bao giờ hoàn hảo, nhưng việc áp dụng nó phải công bằng - đó là điều mà chúng ta đang thiếu, theo ý kiến cá nhân của tôi.

Luật pháp sẽ không bao giờ hoàn hảo, nhưng việc áp dụng nó phải công bằng - đó là điều mà chúng ta đang thiếu, theo ý kiến cá nhân của tôi.

Luật pháp sẽ không bao giờ hoàn hảo, nhưng việc áp dụng nó phải công bằng - đó là điều mà chúng ta đang thiếu, theo ý kiến cá nhân của tôi.

Hình 4: Hai đầu chú ý, cũng ở tầng thứ 5 trong tổng cộng 6 tầng, dường như liên quan đến việc giải quyết vấn đề chỉ dẫn (anaphora resolution). Trên cùng: Toàn bộ chú ý cho đầu chú ý thứ 5. Dưới cùng: Chú ý riêng biệt từ từ 'its' cho hai đầu chú ý thứ 5 và 6. Lưu ý rằng chú ý rất sắc nét đối với từ này.

14



Luật pháp sẽ không bao giờ hoàn hảo, nhưng việc áp dụng nó phải công bằng - đó là

điều mà chúng ta đang thiếu, theo ý kiến cá nhân của tôi.

Luật pháp sẽ không bao giờ hoàn hảo, nhưng việc áp dụng nó phải công bằng - đó là điều mà chúng ta đang thiếu, theo ý kiến cá nhân của tôi.

Luật pháp sẽ không bao giờ hoàn hảo, nhưng việc áp dụng nó phải công bằng - đó là điều mà chúng ta đang thiếu, theo ý kiến cá nhân của tôi.

Luật pháp sẽ không bao giờ hoàn hảo, nhưng việc áp dụng nó phải công bằng - đó là điều mà chúng ta đang thiếu, theo ý kiến cá nhân của tôi.

Hình 5: Nhiều đầu chú ý thể hiện hành vi có vẻ liên quan đến cấu trúc câu. Chúng tôi đưa ra hai ví dụ như vậy ở trên, từ hai đầu khác nhau từ tự chú ý của mã hóa ở tầng thứ 5 trong tổng cộng 6 tầng. Các đầu rõ ràng đã học cách thực hiện các nhiệm vụ khác nhau.

15